Phần III

Khởi động

Trong phần này, chúng ta sẽ làm quen với ba thuật toán machine learning chưa cần nhiều tới tối ưu: K-lân cận cho các bài toán hồi quy và phân loại, K-means cho bài toán phân cụm và bộ phân loại naive Bayes cho dữ liệu dạng văn bản.

Chương 9

K lân cận

Nếu con người có kiểu học "nước đến chân mới nhảy" thì machine learning cũng có một thuật toán như vậy.

9.1. Giới thiệu

9.1.1. K lân cận

K lân cận (*K-nearest neighbor* hay KNN) là một trong những thuật toán học có giám sát đơn giản nhất. Khi huấn luyện, thuật toán này gần như *không học* một điều gì từ dữ liệu huấn luyện mà *ghi nhớ* lại một cách máy móc toàn bộ dữ liệu đó. Mọi tính toán được thực hiện tại pha kiểm tra. KNN có thể được áp dụng vào các bài toán phân loại và hồi quy. KNN còn được gọi là một thuật toán *lười học*, *instance-based* [AKA91], hoặc *memory-based learning*.

KNN là thuật toán đi tìm đầu ra của một điểm dữ liệu mới chỉ dựa trên thông tin của K điểm dữ liệu gần nhất trong tập huấn luyện.

Hình 9.1 mô tả một bài toán phân loại với ba nhãn: đỏ, lam, lục (xem ảnh màu trong Hình B.1). Các hình tròn nhỏ với màu khác nhau thể hiện dữ liệu huấn luyện của các nhãn khác nhau. Các vùng màu nền khác nhau thể hiện "lãnh thổ" của mỗi nhãn. Nhãn của một điểm bất kỳ được xác định dựa trên nhãn của điểm gần nó nhất trong tập huấn luyện. Trong hình này, có một vài vùng nhỏ xem lẫn vào các vùng lớn hơn khác màu. Điểm này rất có thể là nhiễu. Các điểm dữ liệu kiểm tra gần khu vực điểm này nhiều khả năng sẽ bị phân loại sai.

Với KNN, mọi điểm trong tập huấn luyện đều được mô hình mô tả một cách chính xác. Việc này khiến overfitting dễ xảy ra với KNN.

Hình 9.1. Ví dụ về 1NN. Các hình tròn là các điểm dữ liệu huấn luyện. Các hình khác màu thể hiện các lớp khác nhau. Các vùng nền thể hiện các điểm được phân loại vào lớp có màu tương ứng khi sử dụng 1NN (Nguồn: K-nearest neighbors algorithm – Wikipedia, xem ảnh màu trong Hình B.1).

Mặc dù có nhiều hạn chế, KNN vẫn là giải pháp đầu tiên nên nghĩ tới khi giải quyết một bài toán machine learning. Khi làm các bài toán machine learning nói chung, không có mô hình đúng hay sai, chỉ có mô hình cho kết quả tốt hơn. Chúng ta luôn cần một mô hình đơn giản để giải quyết bài toán, sau đó mới dần tìm cách tăng chất lượng của mô hình.

9.2. Phân tích toán học

Không có hàm mất mát hay bài toán tối ưu nào cần thực hiện trong quá trình huấn luyện KNN. Mọi tính toán được tiến hành ở bước kiểm tra. Vì KNN ra quyết định dựa trên các điểm gần nhất nên có hai vấn đề ta cần lưu tâm. Thứ nhất, khoảng cách được định nghĩa như thế nào. Thứ hai, cần phải tính toán khoảng cách như thế nào cho hiệu quả.

Với vấn đề thứ nhất, mỗi điểm dữ liệu được thể hiện bằng một vector đặc trưng, khoảng cách giữa hai điểm chính là khoảng cách giữa hai vector đó. Có nhiều loại khoảng cách khác nhau tuỳ vào bài toán, nhưng khoảng cách được sử dụng nhiều nhất là khoảng cách Euclid (xem Mục 1.14).

Vấn đề thứ hai cần được lưu tâm hơn, đặc biệt với các bài toán có tập huấn luyện lớn và vector dữ liệu có kích thước lớn. Giả sử các vector huấn luyện là các cột của ma trận $\mathbf{X} \in \mathbb{R}^{d \times N}$ với d và N lớn. KNN sẽ phải tính khoảng cách từ một điểm dữ liệu mới $\mathbf{z} \in \mathbb{R}^d$ đến toàn bộ N điểm dữ liệu đã cho và chọn ra K khoảng cách nhỏ nhất. Nếu không có cách tính hiệu quả, khối lượng tính toán sẽ rất lớn.

Tiếp theo, chúng ta cùng thực hiện một vài phân tích toán học để tính các khoảng cách một cách hiệu quả. Ở đây khoảng cách được xem xét là khoảng cách Euclid.

Khoảng cách từ một điểm tới từng điểm trong một tập hợp

Khoảng cách Euclid từ một điểm $\mathbf{z}$ tới một điểm $\mathbf{x}_i$ trong tập huấn luyện được định nghĩa bởi $\|\mathbf{z} - \mathbf{x}_i\|_2$. Người ta thường dùng bình phương khoảng cách Euclid $\|\mathbf{z} - \mathbf{x}_i\|_2^2$ để tránh phép tính căn bậc hai. Việc bình phương này không ảnh hưởng tới thứ tự của các khoảng cách. Để ý rằng

$$\|\mathbf{z}-\mathbf{x}_i\|_2^2 = (\mathbf{z}-\mathbf{x}_i)^T(\mathbf{z}-\mathbf{x}_i) = \|\mathbf{z}\|_2^2 + \|\mathbf{x}_i\|_2^2 - 2\mathbf{x}_i^T\mathbf{z} \tag{9.1}$$

Để tìm ra $\mathbf{x}_i$ gần với $\mathbf{z}$ nhất, số hạng đầu tiên có thể được bỏ qua. Hơn nữa, nếu có nhiều điểm dữ liệu trong tập kiểm tra, các giá trị $\|\mathbf{x}_i\|_2^2$ có thể được tính và lưu trước vào bộ nhớ. Khi đó, ta chỉ cần tính các tích vô hướng $\mathbf{x}_i^T\mathbf{z}$.

Để thấy rõ hơn, chúng ta cùng làm một ví dụ trên Python. Trước hết, chọn d và N là các giá trị lớn và khai báo ngẫu nhiên $\mathbf{X}$ và $\mathbf{z}$. Khi lập trình Python, cần lưu ý rằng chiều thứ nhất thường chỉ thứ tự của điểm dữ liệu.

```
from __future__ import print_function
import numpy as np
from time import time # for comparing runing time
d, N = 1000, 10000 # dimension, number of training points
X = np.random.randn(N, d) # N d-dimensional points
z = np.random.randn(d)
```

Tiếp theo, ta viết ba hàm số:

a. `dist_pp(z, x)` tính bình phương khoảng cách Euclid giữa `z` và `x`. Hàm này tính hiệu $\mathbf{z}-\mathbf{x}$ rồi lấy bình phương ℓ_2 norm của nó.

b. `dist_ps_naive(z, X)` tính bình phương khoảng cách giữa `z` và mỗi hàng của `X`. Trong hàm này, các khoảng cách được xây dựng dựa trên việc tính từng giá trị `dist_pp(z, X[i])`.

c. `dist_ps_fast(z, X)` tính bình phương khoảng cách giữa `z` và mỗi hàng của `X`, tuy nhiên, kết quả được tính dựa trên đẳng thức (9.1). Ta cần tính tổng bình phương các phần tử của mỗi điểm dữ liệu trong `X` và tính tích `X.dot(z)`

Đoạn code dưới đây thể hiện hai cách tính khoảng cách từ một điểm `z` tới một tập hợp điểm `X`. Kết quả và thời gian chạy của mỗi hàm được in ra để so sánh.

```
# naively compute square distance between two vector
def dist_pp(z, x):
    d = z - x.reshape(z.shape) # force x and z to have the same dims
    return np.sum(d*d)

# from one point to each point in a set, naive
def dist_ps_naive(z, X):
    N = X.shape[0]
    res = np.zeros((1, N))
    for i in range(N):
        res[0][i] = dist_pp(z, X[i])
    return res
```

```
# from one point to each point in a set, fast
def dist_ps_fast(z, X):
    X2 = np.sum(X*X, 1) # square of l2 norm of each X[i], can be precomputed
    z2 = np.sum(z*z) # square of l2 norm of z
    return X2 + z2 - 2*X.dot(z) # z2 can be ignored

t1 = time()
D1 = dist_ps_naive(z, X)
print('naive point2set, running time:', time() - t1, 's')

t1 = time()
D2 = dist_ps_fast(z, X)
print('fast point2set , running time:', time() - t1, 's')
print('Result difference:', np.linalg.norm(D1 - D2))
```

Kết quả:

```
naive point2set, running time: 0.0932548046112 s
fast point2set , running time: 0.0514178276062 s
Result difference: 2.11481965531e-11
```

Kết quả chỉ ra rằng hàm `dist_ps_fast(z, X)` chạy nhanh hơn gần gấp đôi so với hàm `dist_ps_naive(z, X)`. Tỉ lệ này còn lớn hơn khi kích thước dữ liệu tăng lên và `X2` được tính từ trước. Quan trọng hơn, sự chênh lệch nhỏ giữa kết quả của hai cách tính chỉ ra rằng `dist_ps_fast(z, X)` nên được ưu tiên hơn.

Khoảng cách giữa từng cặp điểm trong hai tập hợp

Thông thường, tập kiểm tra bao gồm nhiều điểm dữ liệu tạo thành một ma trận $\mathbf{Z}$. Ta phải tính từng cặp khoảng cách giữa mỗi điểm trong tập kiểm tra và một điểm trong tập huấn luyện. Nếu mỗi tập có 1000 phần tử, có một triệu khoảng cách cần tính. Nếu không có phương pháp tính hiệu quả, thời gian thực hiện sẽ rất dài.

Đoạn code dưới đây thể hiện hai phương pháp tính bình phương khoảng cách giữa các cặp điểm trong hai tập điểm. Phương pháp thứ nhất sử dụng một vòng **for** tính khoảng cách từ từng điểm trong tập thứ nhất đến tất cả các điểm trong tập thứ hai thông qua hàm `dist_ps_fast(z, X)` ở trên. Phương pháp thứ hai tiếp tục sử dụng (9.1) cho trường hợp tổng quát.

```
Z = np.random.randn(100, d)
# from each point in one set to each point in another set, half fast
def dist_ss_0(Z, X):
    M, N = Z.shape[0], X.shape[0]
    res = np.zeros((M, N))
    for i in range(M):
        res[i] = dist_ps_fast(Z[i], X)
    return res
```

```
# from each point in one set to each point in another set, fast
def dist_ss_fast(Z, X):
    X2 = np.sum(X*X, 1) # square of l2 norm of each ROW of X
    Z2 = np.sum(Z*Z, 1) # square of l2 norm of each ROW of Z
    return Z2.reshape(-1, 1) + X2.reshape(1, -1) - 2*Z.dot(X.T)

t1 = time()
D3 = dist_ss_0(Z, X)
print('half fast set2set running time:', time() - t1, 's')
t1 = time()
D4 = dist_ss_fast(Z, X)
print('fast set2set  running time', time() - t1, 's')
print('Result difference:', np.linalg.norm(D3 - D4))
```

Kết quả:

```
half fast set2set running time: 4.33642292023 s
fast set2set  running time 0.0583250522614 s
Result difference: 9.93586539607e-11
```

Điều này chỉ ra rằng hai cách tính cho kết quả chênh lệch nhau không đáng kể. Trong khi đó `dist_ss_fast(Z, X)` chạy nhanh hơn `dist_ss_0(Z, X)` nhiều lần.

Khi làm việc trên python, chúng ta có thể sử dụng hàm `cdist` (https://goo.gl/vYMnmM) trong `scipy.spatial.distance`, hoặc hàm `pairwise_distances` (https://goo.gl/QK6Zyi) trong `sklearn.metrics.pairwise`. Các hàm này giúp tính khoảng cách từng cặp điểm trong hai tập hợp khá hiệu quả. Phần còn lại của chương này sẽ trực tiếp sử dụng thư viện scikit-learn cho KNN. Việc viết lại thuật toán này không quá phức tạp khi đã có một hàm tính khoảng cách hiệu quả.

Bạn đọc có thể tham khảo thêm bài báo [JDJ17] về cách thực hiện KNN trên và mã nguồn tại https://github.com/facebookresearch/faiss.

9.3. Ví dụ trên cơ sở dữ liệu Iris

9.3.1. Bộ cơ sở dữ liệu hoa Iris

Bộ dữ liệu hoa Iris (https://goo.gl/eUy83R) là một bộ dữ liệu nhỏ. Bộ dữ liệu này bao gồm thông tin của ba nhãn hoa Iris khác nhau: Iris setosa, Iris virginica và Iris versicolor. Mỗi nhãn chứa thông tin của 50 bông hoa với dữ liệu là bốn thông tin: chiều dài, chiều rộng đài hoa, và chiều dài, chiều rộng cánh hoa. Hình 9.2 là ví dụ về hình ảnh của ba loại hoa. Chú ý rằng các điểm dữ liệu không phải là các bức ảnh mà chỉ là một vector đặc trưng bốn chiếu gồm các thông tin ở trên.

Iris setosa Iris versicolor Iris virginica

Hình 9.2. Ba loại hoa lan trong bộ cơ sở dữ liệu hoa Iris (xem ảnh màu trong Hình B.2).

9.3.2. Thí nghiệm

Trong phần này, 150 điểm dữ liệu được tách thành tập huấn luyện và tập kiểm tra. KNN dựa vào trông tin trong tập huấn luyện để dự đoán mỗi dữ liệu trong tập kiểm tra tương ứng với loại hoa nào. Kết quả này được đối chiếu với đầu ra thực sự để đánh giá hiệu quả của KNN.

Trước tiên, chúng ta cần khai báo vài thư viện. Bộ dữ liệu hoa Iris có sẵn trong thư viện scikit-learn.

```
from __future__ import print_function
import numpy as np
from sklearn import neighbors, datasets
from sklearn.model_selection import train_test_split # for splitting data
from sklearn.metrics import accuracy_score      # for evaluating results

iris = datasets.load_iris()
iris_X = iris.data
iris_y = iris.target
```

Tiếp theo, 20 mẫu dữ liệu được lấy ra ngẫu nhiên tạo thành tập huấn luyện, 130 mẫu còn lại được dùng để kiểm tra.

```
print('Labels:', np.unique(iris_y))

# split train and test
np.random.seed(7)
X_train, X_test, y_train, y_test = train_test_split(
    iris_X, iris_y, test_size=130)
print('Training size:', X_train.shape[0], ', test size:', X_test.shape[0])
```

```
Labels: [0 1 2]
Training size: 20 , test size: 130
```

Dòng `np.random.seed(7)` để đảm bảo kết quả chạy ở các lần khác nhau là giống nhau. Có thể thay 7 bằng một số tự nhiên 32 bit bất kỳ.

Kết quả với 1NN

Tới đây, ta trực tiếp sử dụng thư viện scikit-learn cho KNN. Xét ví dụ đầu tiên với $K = 1$.

```
model = neighbors.KNeighborsClassifier(n_neighbors = 1, p = 2)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy of 1NN: %.2f %%" %(100*accuracy_score(y_test, y_pred)))
```

Kết quả:

```
Accuracy of 1NN: 92.31 %
```

Kết quả thu được là 92.31% (tỉ lệ số mẫu được phân loại chính xác trên tổng số mẫu). Ở đây, `n_neighbors = 1` chỉ ra rằng chỉ điểm gần nhất được lựa chọn, tức $K = 1$, `p = 2` chính là ℓ_2 norm để tính khoảng cách. Bạn đọc có thể thử với `p = 1` tương ứng với khoảng cách ℓ_1 norm.

Kết quả với 7NN

Như đã đề cập, 1NN rất dễ gây ra overfitting. Để hạn chế việc này, ta có thể tăng lượng điểm lân cận lên, ví dụ bảy điểm, kết quả được xác định dựa trên đa số.

```
model = neighbors.KNeighborsClassifier(n_neighbors = 7, p = 2)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy of 7NN with major voting: %.2f %%"\
     %(100*accuracy_score(y_test, y_pred)))
```

Kết quả:

```
Accuracy of 7NN with major voting: 93.85 %
```

Nhận thấy rằng khi sử dụng nhiều điểm lân cận hơn, độ chính xác đã tăng lên. Phương pháp dựa trên đa số trong lân cận còn được gọi là *bầu chọn đa số*.

Đánh trọng số cho các điểm lân cận

Trong kỹ thuật bầu chọn đa số phía trên, các điểm trong bảy điểm gần nhất đều có vai trò như nhau và giá trị "lá phiếu" của mỗi điểm này cũng như nhau. Cách bầu chọn này có thể thiếu công bằng vì các điểm gần hơn nên có tầm ảnh hưởng

lớn hơn. Để thực hiện việc này, ta chỉ cần đánh trọng số khác nhau cho từng điểm trong bảy điểm gần nhất này. Cách đánh trọng số phải thoả mãn điều kiện điểm lân cận hơn được đánh trọng số cao hơn. Một cách đơn giản là lấy nghịch đảo của khoảng cách tới điểm lân cận. Trong trường hợp tồn tại khoảng cách bằng không, tức điểm kiểm tra trùng với một điểm huấn luyện, ta trực tiếp lấy đầu ra của điểm huấn luyện đó.

Để thực hiện việc này trong scikit-learn, ta chỉ cần gán `weights = 'distance'`. Giá trị mặc định của `weights` là `'uniform'`, tương ứng với việc coi tất cả các điểm lân cận có giá trị bằng nhau như trong bầu chọn đa số.

```
model = neighbors.KNeighborsClassifier(n_neighbors = 7, p = 2, \
weights = 'distance')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy of 7NN (1/distance weights): %.2f %%" %(100*accuracy_score(
    y_test, y_pred)))
```

Kết quả:

```
Accuracy of 7NN (1/distance weights): 94.62 %
```

Độ chính xác tiếp tục được tăng lên.

KNN với trọng số tự định nghĩa

Ngoài hai cách đánh trọng số `weights = 'uniform'` và `weights = 'distance'`, scikit-learn còn cung cấp cách đánh trọng số tùy chọn. Ví dụ, một cách đánh trọng số phổ biến khác thường được dùng là

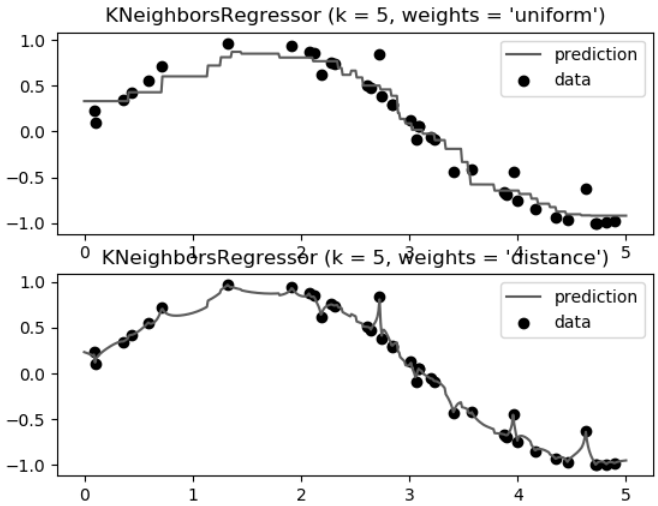
$$w_i = \exp\left(\frac{-\|\mathbf{z} - \mathbf{x}_i\|_2^2}{\sigma^2}\right)$$

trong đó w_i là trọng số của điểm gần thứ i ($\mathbf{x}_i$) của điểm dữ liệu đang xét $\mathbf{z}$, σ là một số dương. Hàm số này cũng thỏa mãn điều kiện điểm càng gần $\mathbf{x}$ thì trọng số càng cao (cao nhất bằng 1). Với hàm số này, ta có thể lập trình như sau:

```
def myweight(distances):
sigma2 = .4 # we can change this number
return np.exp(-distances**2/sigma2)

model = neighbors.KNeighborsClassifier(
    n_neighbors = 7, p = 2, weights = myweight)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy of 7NN (customized weights): %.2f %%"\
      %(100*accuracy_score(y_test, y_pred)))
```



Hình 9.3. KNN cho bài toán hồi quy (Nguồn: *Nearest neighbors regression – scikit-learn* – https://goo.gl/9VyBF3).

Kết quả:

```
Accuracy of 7NN (customized weights): 95.38 %
```

Kết quả tiếp tục tăng lên một chút. Với từng bài toán, chúng ta có thể thay các thuộc tính của KNN bằng các giá trị khác nhau và chọn ra giá trị tốt nhất thông qua xác thực chéo (xem Mục 8.2.2).

9.4. Thảo luận

9.4.1. KNN cho bài toán hồi quy

Với bài toán hồi quy, chúng ta cũng hoàn toàn có thể sử dụng phương pháp tương tự: đầu ra của một điểm được xác định dựa trên đầu ra của các điểm lân cận và khoảng cách tới chúng. Giả sử $\mathbf{x}_1, \ldots, \mathbf{x}_K$ là K điểm lân cận của một điểm dữ liệu $\mathbf{z}$ với đầu ra tương ứng là $y_1, \ldots, y_K$. Giả sử các trọng số ứng với các lân cận này là $w_1, \ldots, w_K$. Kết quả dự đoán đầu ra của $\mathbf{z}$ có thể được xác định bởi

$$\frac{w_1 y_1 + w_2 y_2 + \cdots + w_K w_K}{w_1 + w_2 + \cdots + w_K} \tag{9.2}$$

Hình 9.3 là một ví dụ về KNN cho hồi quy với $K = 5$, sử dụng hai cách đánh trọng số khác nhau. Ta có thể thấy rằng `weights = 'distance'` có xu hướng gây ra overfitting.

9.4.2. Ưu điểm của KNN

- Độ phức tạp tính toán của quá trình huấn luyện gần như bằng 0. Việc tính bình phương ℓ_2 norm của mỗi điểm dữ liệu huấn luyện có thể được thực hiện trước trong bước này.
- Việc dự đoán kết quả của dữ liệu mới tương đối đơn giản sau khi đã xác định được các điểm lân cận.
- KNN không không cần giả sử về phân phối của từng nhãn.

9.4.3. Nhược điểm của KNN

- KNN nhạy cảm với nhiễu khi K nhỏ.
- Khi sử dụng KNN, phần lớn tính toán nằm ở pha kiểm tra. Trong đó việc tính khoảng cách tới từng điểm dữ liệu huấn luyện tốn nhiều thời gian, đặc biệt là với các cơ sở dữ liệu có số chiều lớn và có nhiều điểm dữ liệu. K càng lớn thì độ phức tạp càng cao. Ngoài ra, việc lưu toàn bộ dữ liệu trong bộ nhớ cũng ảnh hưởng tới hiệu năng của KNN.

9.4.4. Đọc thêm

a. *Tutorial To Implement k-Nearest Neighbors in Python From Scratch* (https://goo.gl/J78Qso).

b. Mã nguồn cho chương này có thể được tìm thấy tại https://goo.gl/asF58Q.